

StargazeMPP — Frontend Build Spec

Audience: Frontend developer building the StargazeMPP web app from scratch. **Goal:** A production-ready Next.js 16 app for the agentic intelligence marketplace — provider directory, agent playground, dashboards, wallet-driven voucher signing, dual-rail (Tempo EVM + Solana) support. **Backend contract:** This document assumes the backend exposes the REST + tRPC API described in the companion backend spec.

1. Tech Stack

Concern	Choice	Version	Notes
Framework	Next.js	16.x (App Router)	RSC, Server Actions, Cache Components
Language	TypeScript	5.6+	<code>strict: true, noUncheckedIndexedAccess: true</code>
Styling	Tailwind CSS	4.x	Config-light, JIT
Components	shadcn/ui	latest	Copy-in primitives, no runtime dep
Icons	lucide-react	latest	
Server state	TanStack Query	5.x	All API data
Client state	Zustand	5.x	Wallet/session ephemeral state only
Forms	React Hook Form + Zod	latest	Schema-validated forms
EVM wallet	wagmi + viem	wagmi 2.x, viem 2.x	Tempo L1 connector
Solana wallet	@solana/wallet-adapter	latest	Phantom, Backpack, Solflare
Charts	Recharts	2.x	Reputation/usage charts
Tables	TanStack Table	8.x	Headless, paired with shadcn <code><Table></code>
Date	date-fns	3.x	No moment.js
Markdown (docs)	next-mdx-remote	5.x	<code>/docs</code> content
Animation	framer-motion	11.x	Page transitions, micro-interactions
Testing (unit)	Vitest + Testing Library	latest	
Testing (e2e)	Playwright	latest	Wallet-mocked flows
Lint/format	ESLint 9 + Prettier 3	flat config	
Package manager	pnpm	9.x	Workspaces support
Hosting	Vercel	—	Edge runtime where compatible

2. Project Structure

```
apps/web/
├── app/
│   ├── (marketing)/
│   │   ├── page.tsx          # Landing
│   │   └── layout.tsx
│   ├── explore/
│   │   ├── page.tsx          # Provider directory
│   │   └── _components/
│   ├── providers/
│   │   ├── [id]/
│   │   │   ├── page.tsx      # Provider detail
│   │   │   └── _components/
│   ├── playground/
│   │   ├── page.tsx          # Interactive query tester
│   │   └── _components/
│   ├── dashboard/
│   │   ├── page.tsx          # Agent dashboard
│   │   ├── provider/page.tsx # Provider operator dashboard
│   │   ├── stake/page.tsx    # $GAZE stake/unstake
│   │   └── layout.tsx        # Wallet-gated
│   ├── docs/
│   │   ├── [... slug]/page.tsx # MDX docs
│   │   └── layout.tsx
│   ├── api/
│   │   └── trpc/[trpc]/route.ts # tRPC handler (proxies to backend)
│   ├── layout.tsx            # Root layout, providers
│   ├── error.tsx
│   ├── not-found.tsx
│   └── globals.css
├── components/
│   └── # shadcn primitives (button, card, dialog, ...)
│   ├── ui/
│   ├── marketing/
│   ├── directory/
│   │   ├── provider-card.tsx
│   │   ├── provider-filters.tsx
│   │   └── reputation-badge.tsx
│   ├── playground/
│   │   ├── query-builder.tsx
│   │   ├── response-viewer.tsx
│   │   └── session-monitor.tsx
│   ├── wallet/
│   │   ├── wallet-button.tsx
│   │   ├── rail-switcher.tsx
│   │   └── network-guard.tsx
│   └── shared/
│       ├── price-tag.tsx
│       ├── address-display.tsx
│       └── empty-state.tsx
├── lib/
│   ├── api/          # tRPC client + REST helpers
│   ├── wallet/        # wagmi/solana config
│   ├── voucher/       # EIP-712 voucher signing
│   ├── format/        # currency, address, time formatters
│   ├── hooks/         # custom hooks
│   ├── store/         # Zustand stores
│   ├── env.ts         # env var validation (Zod)
│   ├── styles/
│   │   └── tokens.css  # CSS variables (theme)
│   └── public/
│       └── fonts/
```

```
├── images/
├── tests/
│   ├── unit/
│   └── e2e/
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── package.json
└── .env.example
```

Single Next.js app, not a monorepo on the FE side. Backend lives in a sibling `apps/gateway/` (separate spec).

3. Environment Variables

`.env.example` (validated at boot in `lib/env.ts` via Zod):

```
# Public (exposed to browser)
NEXT_PUBLIC_API_URL=http://localhost:8080
NEXT_PUBLIC_TEMPO_CHAIN_ID=1789 # Tempo L1 mainnet
NEXT_PUBLIC_TEMPO_RPC_URL=https://rpc.tempo.xyz
NEXT_PUBLIC_SOLANA_RPC_URL=https://api.mainnet-beta.solana.com
NEXT_PUBLIC_SESSION_ESCROW_ADDR=0x ...
NEXT_PUBLIC_PROVIDER_REGISTRY_ADDR=0x ...
NEXT_PUBLIC_GAZE_TOKEN_ADDR=0x ...
NEXT_PUBLIC_REPUTATION_ORACLE_ADDR=0x ...
NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID= ...
NEXT_PUBLIC_POSTHOG_KEY=phc_ ...

# Server-only
SENTRY_DSN= ...
INTERNAL_API_TOKEN= ... # for tRPC → backend
```

`lib/env.ts` exports a typed `env` object — fail boot if any required var is missing.

4. Routing & Pages

4.1 / — Landing (RSC)

Purpose: Pitch + live network stats. Funnel to `/explore` and `/docs`.

Sections: - Hero with one-line value prop, CTA buttons (Explore / Read docs). - Live counters: total providers, queries (24h), volume (24h PathUSD). Fetched server-side, revalidated every 60s with `revalidate = 60`. - Three "How it works" cards (Discover / Pay / Earn). - Provider category grid with icons (links into `/explore?category= ...`). - Footer with links to docs, GitHub, status page.

Data: GET `/v1/stats/network` (revalidated).

4.2 `/explore` — Directory (RSC + client filter)

Purpose: Browse all registered providers.

Layout: - Top: search bar (provider id/name), category filter chips, sort dropdown (reputation / latency / price). - Grid of `<ProviderCard>` (3 columns desktop, 1 mobile). - Pagination (20/page) or infinite scroll via `useInfiniteQuery`.

Filtering: - URL is source of truth — `?category=onchain&sort=reputation&q=...`. - Server fetches initial page based on `searchParams`. - Client-side filter changes update URL via `router.push` with `scroll: false`.

Data: `GET /v1/providers?category=&q=&sort=&cursor=`.

Empty state: "No providers match your filters" + reset button.

4.3 `/providers/[id]` — Provider Detail (RSC)

Purpose: Everything an agent or evaluator needs to decide whether to integrate.

Sections: 1. **Header:** logo, name, category badge, verified badge (if `verified: true`), reputation score (gauge), stake amount. 2. **Pricing card:** price per unit, currency, rail (Tempo / Solana / both), estimated cost for 1k queries. 3. **Health card:** uptime 30d, latency p50/p95, error rate. Sparkline of latency over last 24h. 4. **Schema panel:** request schema + response schema (collapsible JSON tree). Use `react-json-view-lite`. 5. **Sample query:** code snippets for cURL, TypeScript (Node), Python — copy buttons. 6. **"Try it"** CTA: deep link into `/playground?provider=<id>`. 7. **Reputation history:** Recharts line chart, weekly score for last 12 weeks.

Data: - `GET /v1/providers/:id` (server) - `GET /v1/providers/:id/health?window=24h` (client, refetch 60s) - `GET /v1/providers/:id/reputation/history` (client)

SEO: generate `metadata` with provider name + description; OG image via `opengraph-image.tsx`.

4.4 `/playground` — Query Tester (Client)

Purpose: Logged-in agent builds and executes a real MPP query end-to-end.

Wallet-gated. If not connected, render `<ConnectPrompt>`.

Layout (3 panels, resizable on desktop): - **Left — Provider selector + request builder.** Dropdown of providers (searchable). Once selected, render Zod-derived form from provider's `requestSchema` (use `@rjsf/core` or hand-rolled form builder). - **Middle — Session monitor.** Shows: active session id, rail, escrow remaining, cumulative spent, query count. Buttons: "Open session", "Close session". - **Right — Response viewer.** Pretty-printed JSON of last response, plus latency, status, voucher signature truncated.

Flow: 1. User clicks "Open session" → opens `<OpenSessionDialog>` (rail picker, deposit amount). 2. User confirms → wagmi `useWriteContract` to `SessionEscrow.openSession` (Tempo) or Solana program call. Backend `POST /v1/sessions` returns `sessionId`. 3. User builds request, clicks "Query". Frontend: - Increments local cumulative voucher amount by provider price. - Signs EIP-712 voucher (Tempo rail) or Ed25519 message (Solana rail) — see §6. - `fetch(POST /v1/query/:providerId, headers: { 'X-MPP-Voucher': sig, 'X-MPP-Session': sessionId, 'X-MPP-Cumulative': cumulative })`. - Renders response or error. 4. User clicks "Close session" → submit final voucher to `POST /v1/sessions/:id/close`. Show settlement status (poll until `settled_tx` populated).

Error handling: - Insufficient escrow → toast "Top up escrow" with prefilled dialog. - Voucher rejected → toast with backend error code. - Provider 5xx → show "Refunded by gateway" with provider id, link to file

reputation report.

4.5 /dashboard — Agent Dashboard (Client, wallet-gated)

Purpose: History and analytics for the connected agent wallet.

Tabs: - **Overview:** stats cards (total spend 30d, queries 30d, providers used, active sessions). Recharts area chart of daily spend. - **Sessions:** TanStack Table of past sessions (id, opened/closed, rail, providers used, total spent, status). Row click → `<SessionDetailDrawer>` showing per-query log. - **Queries:** flat log of all queries, filterable by provider, with response preview on row expand. - **Settings:** notifications (Telegram webhook), default rail, default deposit.

Data: - `GET /v1/agents/me/stats` (TanStack Query, key based on wallet address). - `GET /v1/agents/me/sessions?cursor=` - `GET /v1/agents/me/queries?cursor=&provider=`

Auth: Sign-in-with-Wallet (SIWE for EVM, similar challenge for Solana). Store JWT in httpOnly cookie via Server Action — never in localStorage.

4.6 /dashboard/provider — Provider Operator Dashboard

Purpose: For wallets that operate a registered provider.

Sections: - **Earnings:** stats cards (revenue 30d, queries served 30d, avg latency, refund rate). Chart of daily earnings. - **Reputation:** current score (large gauge), 12-week trend, recent slashing events, recent crowd-vote outcomes. - **Stake:** current stake, ability to top up, withdraw (subject to unbond delay). - **Settings:** update metadata URI, update endpoint URL, pause/unpause provider.

Auth: Same SIWE; backend checks the connected wallet is registered as operator for at least one provider, otherwise show empty state.

4.7 /dashboard/stake — \$GAZE Staking

Purpose: Stake \$GAZE to earn share of routing fees; vote on reputation.

Sections: - Current stake, claimable rewards (PathUSD + USDC). - Stake / unstake forms (validated, slippage-aware not relevant here). - Recent reward distribution events. - Active reputation votes (provider id, vote yes/no, cost: 1 GAZE).

4.8 /docs/[... slug] — Documentation

MDX-based. Sidebar navigation. Code blocks with `shiki` for syntax highlighting. "Edit on GitHub" link. Dark/light theme aware.

Content lives in `content/docs/*.mdx` — file-system routing.

4.9 Error & 404

- `app/error.tsx` — generic error boundary, "Something went wrong" + report button (Sentry capture).
 - `app/not-found.tsx` — 404 with search and link to `/explore`.
-

5. Components

5.1 Primitives (shadcn/ui — install via `npx shadcn@latest add ...`)

`button`, `card`, `dialog`, `dropdown-menu`, `input`, `label`, `select`, `separator`, `sheet`, `table`, `tabs`, `toast`, `tooltip`, `badge`, `skeleton`, `popover`, `command`, `form`, `progress`, `scroll-area`, `accordion`.

5.2 Domain components

`<ProviderCard>`

Props: { `provider: Provider` } **Renders:** logo, name, category, price, latency p95, reputation badge (color-coded), verified checkmark. **Interactions:** entire card is a `<Link>` to `/providers/[id]`. Hover lifts the card.

`<ReputationBadge>`

Props: { `score: number`; `size?: 'sm' | 'md' | 'lg'` } **Renders:** colored pill — green ≥ 90 , yellow 70–89, orange 50–69, red < 50 . Tooltip with breakdown.

`<PriceTag>`

Props: { `amount: bigint`; `currency: 'PathUSD' | 'USDC' | 'GAZE'`; `per?: string` } **Renders:** "0.008 PathUSD / query" — handles bigint formatting with appropriate decimals (PathUSD: 6, USDC: 6, GAZE: 18).

`<AddressDisplay>`

Props: { `address: string`; `chain: 'tempo' | 'solana'`; `truncate?: boolean` } **Renders:** `0x1234...abcd` with copy icon, links to chain explorer in new tab.

`<WalletButton>`

Renders: if disconnected — "Connect wallet" opens `<ConnectModal>` (lists EVM + Solana options). If connected — shows truncated address, current chain badge, dropdown with disconnect, switch network, view on explorer.

`<RailSwitcher>`

Props: { `value: 'tempo' | 'solana'`; `onChange` } **Renders:** segmented control. Disabled options for rails the user has no wallet connected for.

`<NetworkGuard>`

Wraps a tree; if connected wallet is on the wrong chain, shows "Switch to Tempo L1" with a button. Used inside `/playground` and `/dashboard/stake`.

`<QueryBuilder>`

Props: { `schema: JsonSchema`; `value`; `onChange`; `onSubmit` } **Renders:** form generated from JSON schema — text inputs, number inputs, enum selects, nested objects (recursive). Validates on submit.

`<ResponseViewer>`

Props: { response?: unknown; latencyMs?: number; voucherSig?: string; error?: Error }

Renderers: collapsible JSON tree with copy-as-cURL and copy-as-JSON buttons. Shows latency badge and truncated voucher sig.

`<SessionMonitor>`

Props: { sessionId; rail; escrow; cumulative; queryCount } **Renderers:** progress bar (cumulative / escrow), live counters, "Top up" and "Close session" buttons.

`<EmptyState>`

Props: { icon; title; description; cta? } **Renderers:** centered illustration + text + optional CTA. Used for empty lists everywhere.

5.3 Layout components

- `<RootLayout>` : wraps everything in `<Providers>` (TanStack Query, wagmi, Solana wallet, theme, toaster). Includes top nav + footer.
- `<TopNav>` : logo, links (Explore, Playground, Docs), `<WalletButton>` on the right. Mobile: hamburger sheet.
- `<DashboardLayout>` : sidebar nav (Overview, Sessions, Queries, Provider, Stake, Settings) + main content. Wallet-gated — redirects to / if disconnected.
- `<Footer>` : links, social, version.

6. Wallet & Voucher Signing

6.1 Wagmi config (lib/wallet/wagmi.ts)

```
import { createConfig, http } from 'wagmi';
import { injected, walletConnect, coinbaseWallet } from 'wagmi/connectors';
import { defineChain } from 'viem';

export const tempo = defineChain({
  id: Number(process.env.NEXT_PUBLIC_TEMPO_CHAIN_ID),
  name: 'Tempo L1',
  nativeCurrency: { name: 'PathUSD', symbol: 'PathUSD', decimals: 6 },
  rpcUrls: { default: { http: [process.env.NEXT_PUBLIC_TEMPO_RPC_URL!] } },
  blockExplorers: { default: { name: 'TempoScan', url: 'https://tempo.xyz/explorer' } },
});

export const wagmiConfig = createConfig({
  chains: [tempo],
  connectors: [
    injected(),
    walletConnect({ projectId: process.env.NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID! }),
    coinbaseWallet({ appName: 'StargazeMPP' }),
  ],
  transports: { [tempo.id]: http() },
  ssr: true,
});
```


6.2 Solana wallet adapter (lib/wallet/solana.tsx)

Wraps app in `<ConnectionProvider>` + `<WalletProvider>` with adapters for Phantom, Backpack, Solflare. Uses `mainnet-beta` or env-configured RPC.

6.3 Voucher signing (lib/voucher/sign.ts)

EIP-712 voucher for Tempo rail:

```
import { useSignTypedData } from 'wagmi';
import { parseUnits, type Hex } from 'viem';

const DOMAIN = (chainId: number) => ({
  name: 'StargazeMPP',
  version: '1',
  chainId,
  verifyingContract: process.env.NEXT_PUBLIC_SESSION_ESCROW_ADDR! as Hex,
});

const TYPES = {
  Voucher: [
    { name: 'sessionId', type: 'bytes32' },
    { name: 'cumulative', type: 'uint256' },
    { name: 'nonce', type: 'uint64' },
  ],
} as const;

export function useVoucherSigner(chainId: number) {
  const { signTypedDataAsync } = useSignTypedData();
  return async (sessionId: Hex, cumulative: bigint, nonce: bigint) =>
    signTypedDataAsync({
      domain: DOMAIN(chainId),
      types: TYPES,
      primaryType: 'Voucher',
      message: { sessionId, cumulative, nonce },
    });
}
```

For Solana rail: use `signMessage` from `@solana/wallet-adapter-react`, message is canonical JSON `{sessionId, cumulative, nonce}` SHA-256 hashed.

6.4 Session store (Zustand — lib/store/session.ts)

```
type SessionState = {
  sessionId?: `0x${string}`;
  rail?: 'tempo' | 'solana';
  escrow?: bigint;
  cumulative: bigint;
  nonce: bigint;
  queryCount: number;
  open: (s: { sessionId; rail; escrow }) => void;
  recordQuery: (cost: bigint) => void;
  close: () => void;
};
```

Persisted to `sessionStorage` (not local) — sessions are short-lived, shouldn't survive tab close.

7. State Management Strategy

Type of state	Tool	Example
Server data	TanStack Query	provider list, reputation, agent stats
URL state	Next.js <code>searchParams</code> + <code>router.push</code>	filters, sort, pagination
Wallet/connection	wagmi / <code>@solana/wallet-adapter</code> hooks	connected address, chain
Active session	Zustand (sessionStorage-persisted)	sessionId, escrow, cumulative
Form state	React Hook Form	every form
Theme	<code>next-themes</code>	dark/light/system

Never use: Context for derived/server data, Redux, MobX, custom event buses.

8. API Client Layer

8.1 tRPC client (`lib/api/trpc.ts`)

For typed queries that go through the Next.js API route (auth + caching benefits). Used for: agent dashboard data, provider operator data, stake actions.

8.2 Direct fetch (`lib/api/rest.ts`)

For latency-sensitive endpoints called directly from the browser to the backend gateway: `POST /v1/query/:providerId`. Going through Next.js would add a hop — for sub-100ms voucher round-trips this matters.

```
export async function executeQuery(opts: {
  providerId: string;
  body: unknown;
  sessionId: Hex;
  cumulative: bigint;
  voucherSig: Hex;
}) {
  const res = await fetch(`${env.API_URL}/v1/query/${opts.providerId}`, {
    method: 'POST',
    headers: {
      'content-type': 'application/json',
      'x-mpp-session': opts.sessionId,
      'x-mpp-cumulative': opts.cumulative.toString(),
      'x-mpp-voucher': opts.voucherSig,
    },
    body: JSON.stringify(opts.body),
  });
  if (!res.ok) throw await ApiError.fromResponse(res);
  return res.json();
}
```

8.3 Error model

`ApiError` class with `code` (machine-readable), `message`, `status`, `details`. All toast messages map from `code` to a friendly string in `lib/api/error-messages.ts`.

9. Forms & Validation

All forms use React Hook Form + Zod resolver. Reusable `<Form>`, `<FormField>`, `<FormItem>`, `<FormControl>`, `<FormMessage>` from shadcn.

Example — `<OpenSessionForm>`:

```
const schema = z.object({
  rail: z.enum(['tempo', 'solana']),
  deposit: z.coerce.number().positive().max(10_000),
});
```

Numeric inputs that represent on-chain amounts must round-trip to bigint via `parseUnits` — never store as `number` past form validation.

10. Loading, Error, Empty States

- Every list view: skeleton placeholders (shadcn `<Skeleton>`) matching final layout while loading.
 - Every fetch: error state with retry button, captured to Sentry.
 - Every list: empty state component.
 - Suspense boundaries around large RSC sections so streaming works.
-

11. Accessibility

- All interactive elements keyboard-navigable, visible focus rings (Tailwind `focus-visible:`).
 - All images have `alt`. Decorative images: `alt=""`.
 - Color contrast: AAA for body text, AA for everything else. Don't rely on color alone (reputation badges have icons too).
 - Forms: every input has a `<label>`; errors announced via `aria-describedby`.
 - Modals trap focus, return focus on close.
 - Live regions (`aria-live="polite"`) for toasts and session updates.
 - Test with axe-core via `@axe-core/playwright` in e2e suite.
-

12. Performance

- **Code splitting:** automatic per-route via Next.js. Dynamic-import heavy components: JSON viewer, charts, code highlighter.

- **Images:** `next/image` for all images. SVG inline for icons.
 - **Fonts:** `next/font/local` for self-hosted fonts (Inter Variable). No FOUT.
 - **Bundle budget:** total client JS per route $\leq$ 200KB gzipped. Track with `@next/bundle-analyzer` in CI.
 - **RSC by default.** Only mark `'use client'` when interactivity required. Push state down — small client components, large server trees.
 - **Cache headers:** static pages `revalidate = 60`, immutable assets via Next defaults.
 - **Prefetch:** Next link prefetching default-on for in-viewport links; opt out for low-value pages.
 - **Lighthouse target:** ≥ 95 on Performance, Accessibility, Best Practices, SEO for landing + explore + provider detail.
-

13. Theme & Design System

CSS variables in `styles/tokens.css`. Tailwind config consumes them.

Color tokens:

```
:root {
  --color-bg:      #0a0a0f;      /* deep dark */
  --color-surface: #14141c;
  --color-border:  #2a2a3a;
  --color-text:    #e8e8f0;
  --color-text-dim: #9090a8;
  --color-primary: #c026d3;      /* magenta - matches PDF brand */
  --color-primary-fg: #ffffff;
  --color-success: #22c55e;
  --color-warn:     #f59e0b;
  --color-danger:   #ef4444;
  --color-accent:   #84cc16;     /* lime accent for stats/highlights */
}
```

Dark mode is default. Light mode optional (toggle in footer). Use `next-themes` with `class` strategy.

Typography: - Display: Inter Variable, 700, tracking-tight. - Body: Inter Variable, 400. - Mono: Cascadia Code, JetBrains Mono fallback. - Scale: 12 / 14 / 16 / 18 / 20 / 24 / 30 / 36 / 48 / 60.

Spacing: Tailwind default (4px base).

Radius: 6 (sm), 10 (md), 14 (lg). All tailwind tokens.

Shadows: subtle. Primary glow on focus and on `<ProviderCard>` hover.

14. Testing

Unit (Vitest + Testing Library)

- All formatters in `lib/format/`.
- Zod schemas — both happy and error paths.
- Voucher signing logic with viem mocks.
- Components: `<ProviderCard>`, `<PriceTag>`, `<ReputationBadge>`, form components.

Coverage target: $\geq 80\%$ for `Lib/`, $\geq 60\%$ overall.

E2E (Playwright)

Test users via mocked wallet (use `@synthetixio/synpress` or custom `window.ethereum` shim).

Critical flows: 1. Connect wallet $\rightarrow$ browse `/explore` $\rightarrow$ open provider detail $\rightarrow$ click "Try it" $\rightarrow$ land in playground with provider preselected. 2. Open session $\rightarrow$ execute query $\rightarrow$ close session $\rightarrow$ confirm settlement appears in dashboard. 3. Insufficient escrow path $\rightarrow$ top-up dialog appears $\rightarrow$ top-up succeeds $\rightarrow$ query retries. 4. Disconnect wallet from `/dashboard` $\rightarrow$ redirected to `/`. 5. Stake \$GAZE $\rightarrow$ claim rewards $\rightarrow$ unstake.

Run on PR via GitHub Actions.

Visual regression (optional Phase 2)

Chromatic or Percy on Storybook.

15. Observability

- **Errors:** Sentry, sourcemaps uploaded on Vercel build.
- **Analytics:** PostHog — page views, key events (`wallet_connected`, `session_opened`, `query_executed`, `session_closed`, `stake_added`).
- **Web vitals:** `next/web-vitals` $\rightarrow$ PostHog event.
- **No PII** logged. Wallet addresses are pseudonymous; do not log query payloads (may contain sensitive params).

16. Build, CI, Deploy

Local

```
pnpm install
pnpm dev      # next dev with turbopack on :3000
pnpm test     # vitest
pnpm test:e2e # playwright (assumes backend running locally)
pnpm lint     # eslint + prettier check
pnpm typecheck # tsc --noEmit
pnpm build    # next build
```

CI (GitHub Actions, on every PR)

1. Install (cached pnpm store)
2. Lint + typecheck (parallel)
3. Unit tests with coverage upload
4. Build
5. E2E against built output + ephemeral backend (docker-compose)

6. Bundle size check (fail if any route > 200KB gzipped)
7. Lighthouse CI on landing + explore + provider detail (fail if Performance < 90)

Deploy

Vercel project. Preview deploy per PR. Production deploy on merge to `main`. Env vars set per environment in Vercel dashboard. Rollback via Vercel UI or `vercel rollback`.

17. Setup Instructions (for the new dev)

1. Install Node 22, pnpm 9, git.
2. Clone repo, `cd apps/web`.
3. `pnpm install`.
4. Copy `.env.example` → `.env.local`. Get values from team password manager.
5. Start backend (see backend spec) — required for `/playground` and `/dashboard`.
6. `pnpm dev`. Open `http://localhost:3000`.
7. Install browser wallet extension (Rabby / MetaMask) and add Tempo L1 from `lib/wallet/wagmi.ts`.
8. Get test PathUSD from team faucet.
9. Run through e2e checklist in `tests/e2e/smoke.spec.ts`.

18. Build Order — 8 Weeks

Week	Deliverable
1	Repo scaffold, CI green, tokens.css + Tailwind, root layout, <code><TopNav></code> , <code><WalletButton></code> , env validation
2	<code>/landing</code> , <code>/explore</code> with mock data, <code><ProviderCard></code> , <code><ReputationBadge></code> , <code><PriceTag></code>
3	<code>/providers/[id]</code> detail with charts and schema viewer, real backend integration for directory
4	<code>/playground</code> skeleton, <code><QueryBuilder></code> from JSON schema, <code><ResponseViewer></code>
5	Voucher signing, <code><SessionMonitor></code> , full playground happy path against real backend
6	<code>/dashboard</code> (agent), SIWE auth, <code><SessionDetailDrawer></code> , all data tables
7	<code>/dashboard/provider</code> , <code>/dashboard/stake</code> , all forms wired
8	<code>/docs</code> MDX system, Lighthouse pass, e2e suite green, accessibility audit, Sentry+PostHog wired

19. Out of Scope

- Mobile native apps (web is responsive; native deferred).
- i18n (English only at launch).
- White-label theming for partners.
- In-app provider registration UX (operators register via CLI in v1; UI in Phase 2).

- Real-time WebSocket subscriptions (poll-based for v1).
- Confidential transaction UX (waits on Tempo confidential-tx GA).